

Blockchain Experiment 5

Name: Atharva Lotankar

Class: D20C **R.No.:** 20

Date: 11 / 09 / 2026

Aim: Study of solidity program structure, basic constructs compilation and deployment environment

Theory:

1. Primitive Data Types, Variables, Functions – pure, view

In Solidity, primitive data types form the foundation of smart contract development. Commonly used types include:

- **uint / int:** unsigned and signed integers of different sizes (e.g., uint256, int128).
- **bool:** represents logical values (true or false).
- **address:** holds a 20-byte Ethereum account address, often used for storing user accounts or contract addresses.
- **bytes / string:** store binary data or textual data.

Variables in Solidity can be **state variables** (stored on the blockchain permanently), **local variables** (temporary, created during function execution), or **global variables** (special predefined variables such as msg.sender, msg.value, and block.timestamp).

Functions allow execution of contract logic. Special types of functions include:

- **pure:** cannot read or modify blockchain state; they work only with inputs and internal computations.
- **view:** can read state variables but cannot alter them. This classification helps optimize gas usage and enforces function integrity.

2. Inputs and Outputs to Functions

Functions in Solidity can accept input arguments and return one or more output values. Inputs enable users or other contracts to pass data into the contract, while outputs make it possible to return results after computation. For example, a function can accept an amount in Ether and return whether the transfer was successful. Solidity also allows named return variables, which improve readability and debugging.

3. Visibility, Modifiers and Constructors

- **Function Visibility** defines who can access a function:
 - **public**: available both inside and outside the contract.
 - **private**: only accessible within the same contract.
 - **internal**: accessible within the contract and its child contracts.
 - **external**: can be called only by external accounts or other contracts.
- **Modifiers** are reusable code blocks that change the behavior of functions. They are often used for access control, such as restricting sensitive functions to the contract owner (`onlyOwner`).
- **Constructors** are special functions executed only once during contract deployment. They initialize important values, such as setting the deploying account as the owner of the contract.

4. Control Flow: if-else, loops

Control flow in Solidity is similar to traditional programming languages:

- **if-else** allows conditional decision-making in contract logic, e.g., checking if a balance is sufficient before transferring funds.
- **Loops** (`for`, `while`, `do-while`) enable repeated execution of code. For example, iterating through an array of users. However, loops must be used carefully, as excessive iterations increase gas consumption, potentially making the contract expensive to execute.

5. Data Structures: Arrays, Mappings, Structs, Enums

- **Arrays**: Can be fixed or dynamic and are used to store ordered lists of elements. Example: an array of addresses for registered users.
- **Mappings**: Key-value pairs that allow quick lookups. Example: `mapping(address => uint)` for storing balances. Unlike arrays, mappings do not support iteration.
- **Structs**: Allow grouping of related properties into a single data type, such as creating a struct `Player {string name; uint score;}`.
- **Enums**: Used to define a set of predefined constants, making code more readable. Example: `enum Status { Pending, Active, Closed }`.

6. Data Locations

Solidity uses three primary data locations for storing variables:

- **storage:** Data stored permanently on the blockchain. Examples: state variables.
- **memory:** Temporary data storage that exists only while a function is executing. Used for local variables and function inputs.
- **calldata:** A non-modifiable and non-persistent location used for external function parameters. It is gas-efficient compared to memory. Understanding data locations is essential, as they directly impact gas costs and performance.

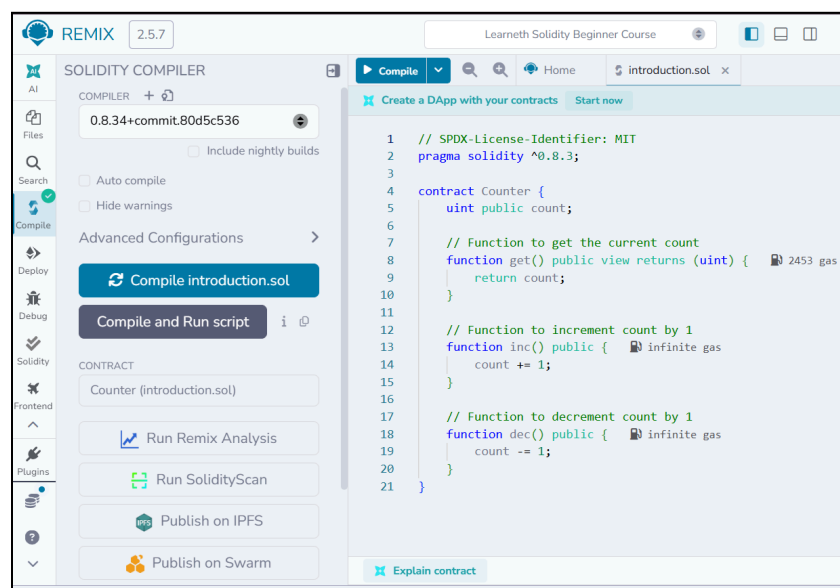
7. Transactions: Ether and Wei, Gas and Gas Price, Sending Transactions

- **Ether and Wei:** Ether is the main currency in Ethereum. All values are measured in Wei, the smallest unit (1 Ether = 10^{18} Wei). This ensures high precision in financial transactions.
- **Gas and Gas Price:** Every transaction consumes gas, which represents computational effort. The gas price determines how much Ether is paid per unit of gas. A higher gas price incentivizes miners to prioritize the transaction.
- **Sending Transactions:** Transactions are used for transferring Ether or interacting with contracts. Functions like `transfer()` and `send()` are commonly used, while `call()` provides more flexibility. Each transaction requires gas, making efficiency in contract design very important.

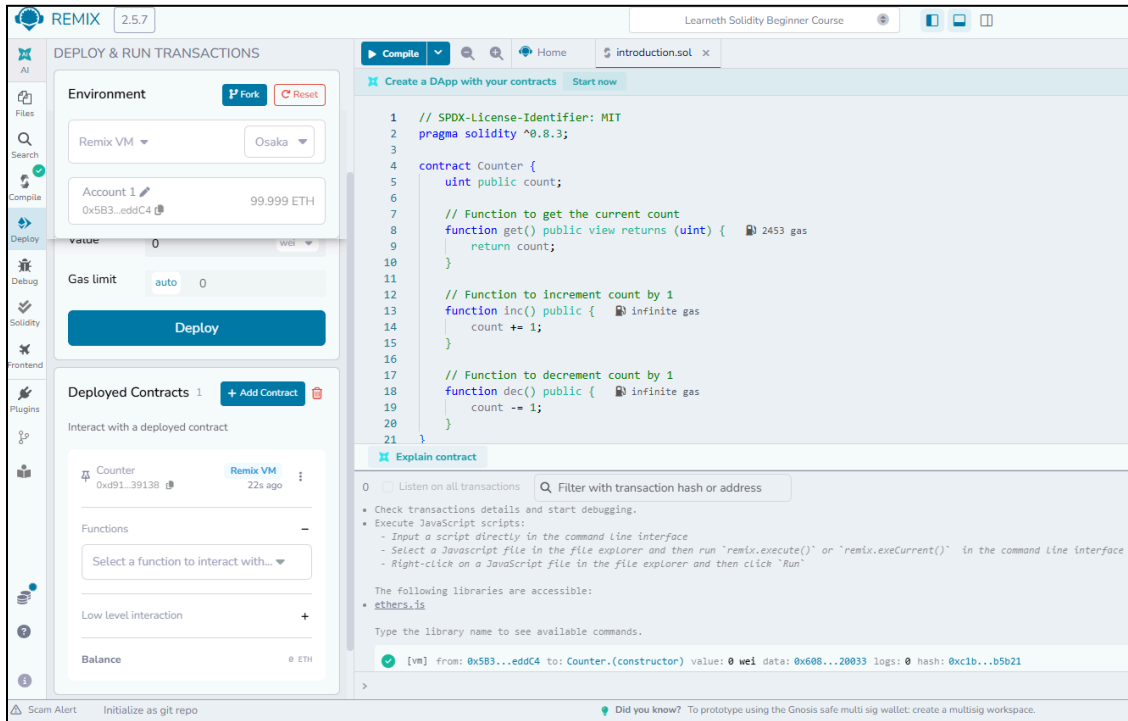
Implementation:

Tutorial 1: Introduction to Solidity

- *Compile This Contract*

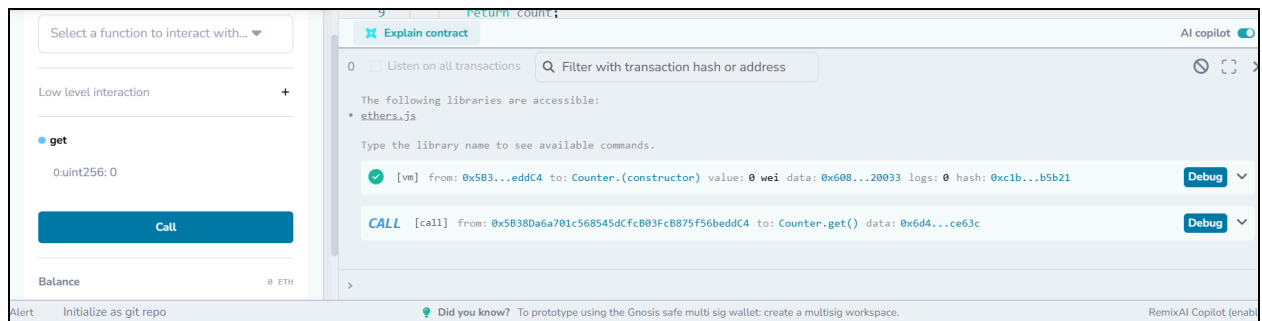


- *Deploy it to the Remix VM.*

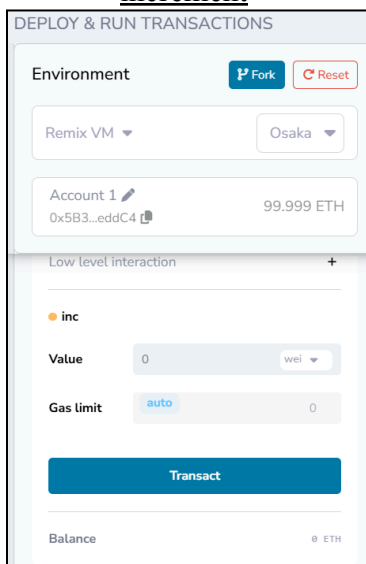


- *Interact with your contract*

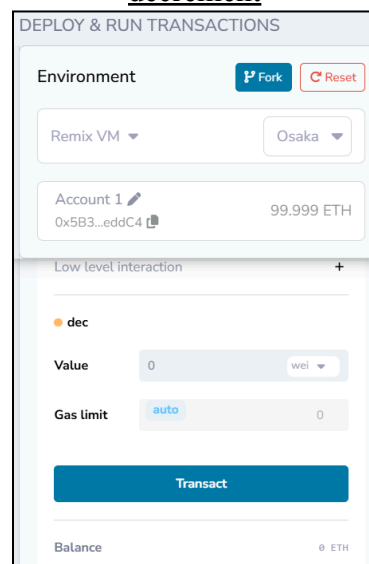
get

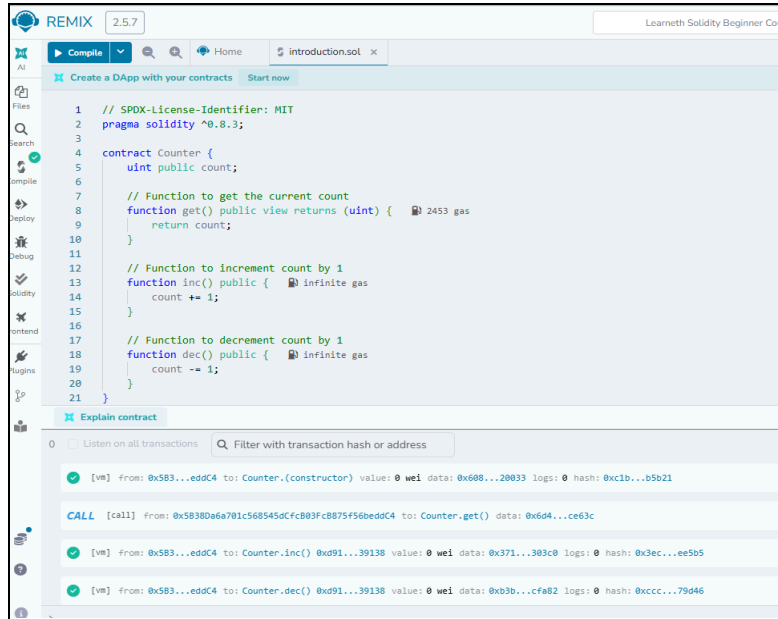


increment

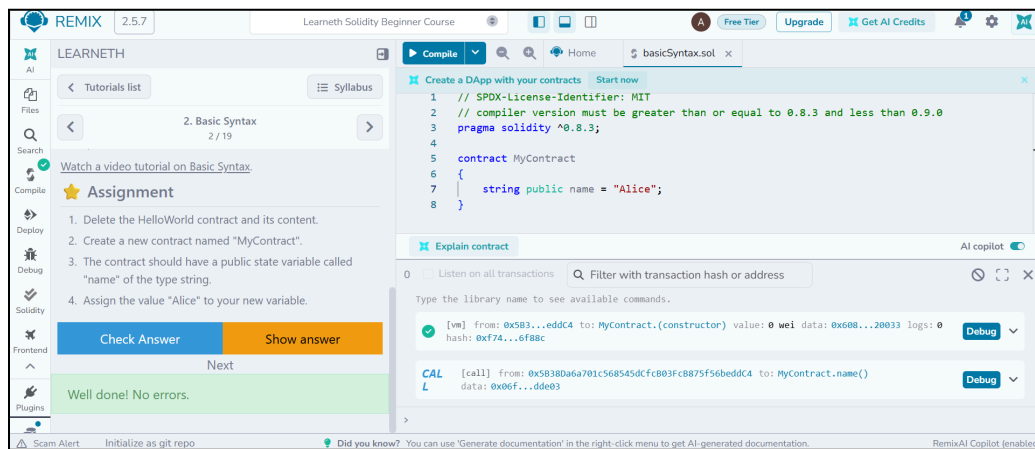


decrement

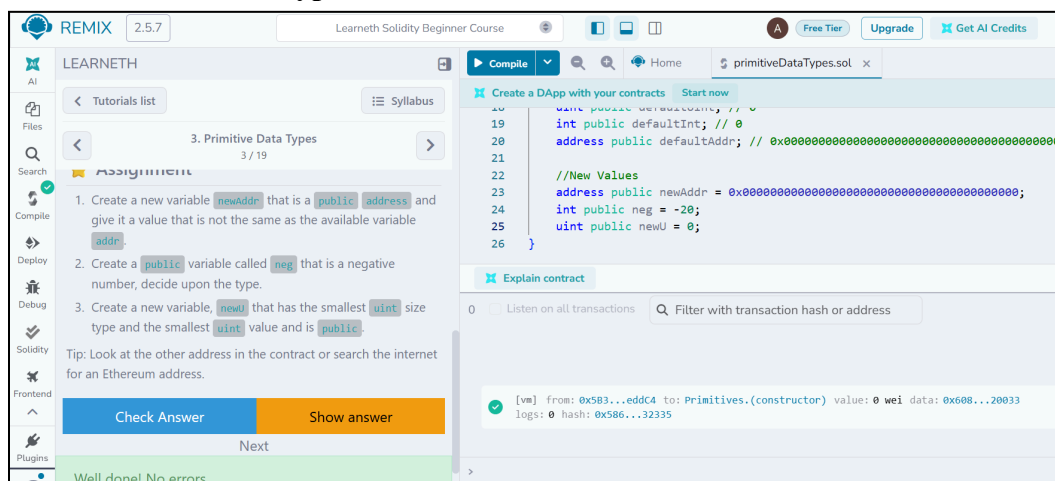




Tutorial 2: Basic Syntax



Tutorial 3: Primitive Data Types



Tutorial 4: Variables

LEARNETH

4. Variables
4 / 19

Watch video tutorials on State Variables, Local Variables, and Global Variables.

Assignment

1. Create a new public state variable called `blockNumber`.
2. Inside the function `doSomething()`, assign the value of the current block number to the state variable `blockNumber`.

Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.

Check Answer Show answer

Next

Well done! No errors.

```
5 // State variables are stored on the blockchain.
6 string public text = "Hello";
7 uint public num = 123;
8 uint public blockNumber;
9
10 function doSomething() public {
11     // Local variables are not saved to the blockchain.
12     uint i = 456;
13
14     // Here are some global variables
15     uint timestamp = block.timestamp; // Current block timestamp
16     address sender = msg.sender; // address of the caller
17     blockNumber = block.number;
18 }
19 }
```

Explain contract

0 Listen on all transactions Filter with transaction hash or address

[vm] from: 0x5B3...eddC4 to: Primitives.constructor value: 0 wei data: 0x608...20033 logs: 0 hash: 0x586...32335

[vm] from: 0x5B3...eddC4 to: Variables.constructor value: 0 wei data: 0x608...20033 logs: 0 hash: 0x106...ba581

Tutorial 5: Functions - Reading & Writing to a State Variable

LEARNETH

5.1 Functions - Reading and Writing to a State Variable
5 / 19

have to specify the return types. In this case, it's a `uint` since the state variable `num` that the function returns is a `uint`.

We will explore the particularities of Solidity functions in more detail in the following sections.

Watch a video tutorial on Functions.

Assignment

1. Create a public state variable called `b` that is of type `bool` and initialize it to `true`.
2. Create a public function called `get_b` that returns the value of `b`.

Check Answer Show answer

Next

Well done! No errors.

```
5 // State variable to store a number
6 uint public num;
7 bool public b = true;
8
9 // You need to send a transaction to write to a state variable.
10 function set(uint _num) public {
11     num = _num;
12 }
13
14 // You can read from a state variable without sending a transaction.
15 function get() public view returns (uint) {
16     return num;
17 }
18 function get_b() public view returns (bool) {
19     return b;
20 }
```

Explain contract

0 Listen on all transactions Filter with transaction hash or address

[vm] from: 0x5B3...eddC4 to: SimpleStorage.constructor value: 0 wei data: 0x608...20033 logs: 0 hash: 0x4f6...85a07

Tutorial 6: Functions - View and Pure

REMIX 2.5.7

Learneth Solidity Beginner Course

LEARNETH

5.2 Functions - View and Pure
6 / 19

In Solidity development, you need to optimise your code for saving computation cost (gas cost). Declaring functions view and pure can save gas cost and make the code more readable and easier to maintain. Pure functions don't have any side effects and will always return the same result if you pass the same arguments.

Watch a video tutorial on View and Pure Functions.

Assignment

Create a function called `addToX2` that takes the parameter `y` and updates the state variable `x` with the sum of the parameter and the state variable `x`.

Check Answer Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract ViewAndPure {
5     uint public x = 1;
6
7     // Promise not to modify the state.
8     function addToX(uint y) public view returns (uint) {
9         return x + y;
10    }
11
12    // Promise not to modify or read from the state.
13    function add(uint i, uint j) public pure returns (uint) {
14        return i + j;
15    }
16
17    function addToX2 (uint y) public {
18        x = x + y;
19    }
20 }
```

Explain contract

0 Listen on all transactions Filter with transaction hash or address

Tutorial 7: Functions - Modifiers and Constructors

5.3 Functions - Modifiers and Constructors
7 / 19

parameters and is especially useful when you don't know certain initialization values before the deployment of the contract.

You declare a constructor using the `constructor` keyword. The constructor in this contract (line 11) sets the initial value of the owner variable upon the creation of the contract.

[Watch a video tutorial on Function Modifiers.](#)

Assignment

1. Create a new function, `increaseX` in the contract. The function should take an input parameter of type `uint` and increase the value of the variable `x` by the value of the input parameter.
2. Make sure that `x` can only be increased.
3. The body of the function `increaseX` should be empty.

Tip: Use modifiers.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

```
46
47
48 function increaseX(uint y) public onlyOwner biggerThan0(y) increaseXByY(y){
49 }
50
51 modifier noReentrancy() {
52     require(!locked, "No reentrancy");
53
54     locked = true;
55     _;
56     locked = false;
57 }
58
59 function decrement(uint i) public noReentrancy {
60     x -= i;
61
62     if (i > 1) {
63         decrement(i - 1);
64     }
65 }
```

[Explain contract](#)

0 ☐ Listen on all transactions

[vm] from: 0x583...eddC4 to: FunctionModifier.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0x4a3...3881e

Tutorial 8: Functions - Inputs and Outputs

5.4 Functions - Inputs and Outputs
8 / 19

"[Mappings] cannot be used as parameters or return parameters of contract functions that are publicly visible." From the [Solidity documentation](#).

Arrays can be used as parameters, as shown in the function `arrayInput` (line 71). Arrays can also be used as return parameters as shown in the function `arrayOutput` (line 76).

You have to be cautious with arrays of arbitrary size because of their gas consumption. While a function using very large arrays as inputs might fail when the gas costs are too high, a function using a smaller array might still be able to execute.

[Watch a video tutorial on Function Outputs.](#)

Assignment

Create a new function called `returnTwo` that returns the values `-2` and `true` without using a return statement.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

```
73
74 // Can use array for output
75 uint[] public arr;
76
77 function arrayOutput() public view returns (uint[] memory) {
78     return arr;
79 }
80
81 function returnTwo()
82     public
83     pure
84     returns (
85         int i,
86         bool b
87     )
88 {
89     i = -2;
90     b = true;
91 }
```

[Explain contract](#)

0 ☐ Listen on all transactions

[vm] from: 0x583...eddC4 to: Function.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0xab1...b0703

Tutorial 9: Visibility

6. Visibility
9 / 19

function `privateFunc` from the `Base` contract.

If you compile and deploy the two contracts, you will not be able to call the functions `privateFunc` and `internalFunc` directly. You will only be able to call them via `testPrivateFunc` and `testInternalFunc`.

[Watch a video tutorial on Visibility.](#)

Assignment

Create a new function in the `Child` contract called `testInternalVar` that returns the values of all state variables from the `Base` contract that are possible to return.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

```
58 // Base contract: Base.sol
59 //
60 //
61
62 // Internal function call be called inside child contracts.
63 function testInternalFunc() public pure override returns (string memory) {
64     return internalFunc();
65 }
66
67 //Child Contract
68 function testInternalVar() public view returns (string memory, string memory) {
69     return (internalVar, publicVar);
70 }
```

[Explain contract](#)

0 ☐ Listen on all transactions

[vm] from: 0x583...eddC4 to: Base.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0xab1...b043d

Tutorial 10: Control Flow - If/Else

REMIX 2.5.7 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

7.1 Control Flow - If/Else 10 / 19

Watch a video tutorial on the If/Else statement.

★ Assignment

Create a new function called `evenCheck` in the `IfElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternary operator to return the result of the function.

Tip: The modulo (%) operator produces the remainder of an integer division.

Check Answer Show answer

Next

Well done! No errors.

```
18 // }
19 // return 2;
20
21 // shorthand way to write if / else statement
22 return _x < 10 ? 1 : 2;
23 }
24
25 function evenCheck (uint b) public pure returns (bool){ infinite gas
26     return b % 2 == 0 ? true : false;
27 }
28 }
```

Explain contract

0 Listen on all transactions Filter with transaction hash or address

[vm] from: 0x5B3...eddC4 to: IfElse.(constructor) value: 0 wei data: 0x608...20033 logs: 0 has

Tutorial 11: Control Flow - Loops

REMIX 2.5.7 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

7.2 Control Flow - Loops 11 / 19

statement (line 10) will prevent the second if statement (line 12) from being executed.

break

The `break` statement is used to exit a loop. In this contract, the break statement (line 14) will cause the for loop to be terminated after the sixth iteration.

Watch a video tutorial on Loop statements.

★ Assignment

- Create a public `uint` state variable called `count` in the `Loop` contract.
- At the end of the for loop, increment the count variable by 1.
- Try to get the count variable to be equal to 9, but make sure you don't edit the `break` statement.

Check Answer Show answer

Next

Well done! No errors.

```
4 contract Loop {
5     uint public count;
6     function loop() public { infinite gas
7         // for loop
8         for (uint i = 0; i < 10; i++) {
9             if (i == 5) {
10                // Skip to next iteration with continue
11                continue;
12            }
13            if (i == 5) {
14                // Exit loop with break
15                break;
16            }
17            count++;
18        }
19    }
20    // while loop
21    uint j;
22    while (j < 10) {
```

Explain contract

0 Listen on all transactions Filter with transaction hash or address

[vm] from: 0x5B3...eddC4 to: Loop.(constructor) value: 0 wei data: 0x608...

Tutorial 12: Data Structures - Arrays

LEARNETH

Tutorials list

8.1 Data Structures - Arrays 12 / 19

index from an array (line 42). When we remove an element with the `delete` operator all other elements stay the same, which means that the length of the array will stay the same. This will create a gap in our array. If the order of the array is not important, then we can move the last element of the array to the place of the deleted element (line 46), or use a mapping. A mapping might be a better choice if we plan to remove elements in our data structure.

Array length

Using the length member, we can read the number of elements that are stored in an array (line 35).

Watch a video tutorial on Arrays.

★ Assignment

- Initialize a public fixed-sized array called `arr3` with the values 0, 1, 2. Make the size as small as possible.
- Change the `getArr()` function to return the value of `arr3`.

Check Answer Show answer

Next

Well done! No errors.

```
4 contract Array {
5     // Several ways to initialize an array
6     uint[] public arr;
7     uint[] public arr2 = [1, 2, 3];
8     // Fixed sized array, all elements initialize to 0
9     uint[10] public myFixedSizeArr;
10    uint[3] public arr3 = [0, 1, 2];
11
12    function get(uint i) public view returns (uint) { infinite gas
13        return arr[i];
14    }
15
16    // Solidity can return the entire array.
17    // But this function should be avoided for
18    // arrays that can grow indefinitely in length.
19    function getArr() public view returns (uint[3] memory) { infinite gas
20        return arr3;
21    }
22
23    function push(uint i) public { 46820 gas
24        // Append to array
25        // This will increase the array length by 1.
```

Explain contract

0 Listen on all transactions Filter with transaction hash or address

[vm] from: 0x5B3...eddC4 to: Array.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0xb48...1e264

Tutorial 13: Data Structures - Mappings

8.2 Data Structures - Mappings 13 / 19

We set a new value for a key by providing the mapping's name and key in brackets and assigning it a new value (line 16).

Removing values

We can use the delete operator to delete a value associated with a key, which will set it to the default value of 0. As we have seen in the arrays section.

Watch a video tutorial on Mappings.

★ **Assignment**

1. Create a public mapping `balances` that associates the key type `address` with the value type `uint`.
2. Change the functions `get` and `remove` to work with the mapping `balances`.
3. Change the function `set` to create a new entry to the `balances` mapping, where the key is the address of the parameter and the value is the balance associated with the address of the parameter.

Check Answer Show answer

Next

Well done! No errors.

```
3
4 contract Mapping {
5     // Mapping from address to uint
6     mapping(address => uint) public balances;
7
8     function get(address _addr) public view returns (uint) {
9         // Mapping always returns a value.
10        // If the value was never set, it will return the default value.
11        return balances[_addr];
12    }
13
14    function set(address _addr) public {
15        // Update the value at this address
16        balances[_addr] = _addr.balance;
17    }
18
19    function remove(address _addr) public {
20        // Reset the value to the default value.
21        delete balances[_addr];
22    }
23
24
25    contract NestedMapping {
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```
function get(address _addr1, uint _i) public view returns (bool) {
    // You can get values from a nested mapping
    // even when it is not initialized
    return nested[_addr1][_i];
}

function set(
    address _addr1,
    uint _i,
    bool _boo
) public {
    nested[_addr1][_i] = _boo;
}

function remove(address _addr1, uint _i) public {
    delete nested[_addr1][_i];
}
```

Tutorial 14: Data Structures - Structs

8.3 Data Structures - Structs 14 / 19

and values as a mapping inside curly braces (line 19).

Initialize and update a struct: We initialize an empty struct first and then update its member by assigning it a new value (line 23).

Accessing structs

To access a member of a struct we can use the dot operator (line 33).

Updating structs

To update a struct's member we also use the dot operator and assign it a new value (lines 39 and 45).

Watch a video tutorial on Structs.

★ **Assignment**

Create a function `remove` that takes a `uint` as a parameter and deletes a struct member with the given index in the `todos` mapping.

Check Answer Show answer

Next

Well done! No errors.

```
36
37 // update text
38 function update(uint _index, string memory _text) public {
39     Todo storage todo = todos[_index];
40     todo.text = _text;
41 }
42
43 // update completed
44 function toggleCompleted(uint _index) public {
45     Todo storage todo = todos[_index];
46     todo.completed = !todo.completed;
47 }
48
49 function remove(uint _index) public {
50     delete todos[_index];
51 }
```

Tutorial 15: Data Structures - Enums

8.4 Data Structures - Enums 15 / 19

We can use the delete operator to delete the enum value of the variable, which means as for arrays and mappings, to set the default value to 0.

Watch a video tutorial on Enums.

★ **Assignment**

1. Define an enum type called `Size` with the members `S`, `M`, and `L`.
2. Initialize the variable `sizes` of the enum type `Size`.
3. Create a getter function `getSize()` that returns the value of the variable `sizes`.

Check Answer Show answer

Next

Well done! No errors.

```
11
12
13 }
14
15 enum Size {
16     S,
17     M,
18     L
19 }
20
21 Status public status;
22 Size public sizes;
23
24 function get() public view returns (Status) {
25     return status;
26 }
27
28 function getSize() public view returns (Size) {
29     return sizes;
30 }
```

Tutorial 16: Data Locations

LEARNETH 2.5.7 | Learneth Solidity Beginner Course

9. Data Locations 16 / 19

Assignment

1. Change the value of the `myStruct` member `foo`, inside the `function` `f`, to 4.
2. Create a new struct `myNewStruct2` with the data location `memory` inside the `function f` and assign it the value of `myStruct`. Change the value of the `myNewStruct2` member `foo` to 1.
3. Create a new struct `myNewStruct3` with the data location `memory` inside the `function f` and assign it the value of `myStruct`. Change the value of the `myNewStruct3` member `foo` to 3.
4. Let the function `f` return `myStruct`, `myNewStruct2`, and `myNewStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

Check Answer **Show answer**

Next

Well done! No errors.

dataLocations.sol

```
17 myStruct storage myStruct = myStruct(4);
18 myStruct.foo = 4;
19 // create a struct in memory
20 MyStruct memory myNewStruct = MyStruct(0);
21 MyStruct memory myNewStruct2 = myNewStruct;
22 myNewStruct2.foo = 1;
23
24 MyStruct memory myNewStruct3 = myStruct;
25 myNewStruct3.foo = 3;
26 return (myStruct, myNewStruct2, myNewStruct3);
27
28 function f() {
29     uint[] storage _arr;
30     mapping(uint => address) storage _map;
31     MyStruct storage _myStruct
32 } internal {
33     // do something with storage variables
34 }
```

Explain contract

0 ☐ Listen on all transactions

[vm] from: 0x5B3...eddC4 to: DataLocations.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0x8cb...63999

Tutorial 17: Ether and Wei

LEARNETH 2.5.7 | Learneth Solidity Beginner Course

10.1 Transactions - Ether and Wei 17 / 19

One `gwei` (giga-wei) is equal to 1,000,000,000 (10^9) `wei`.

One `ether` is equal to 1,000,000,000,000,000 (10^{18}) `wei` (line 11).

Watch a video tutorial on Ether and Wei.

Assignment

1. Create a `public uint` called `oneGwei` and set it to 1 `gwei`.
2. Create a `public bool` called `isOneGwei` and set it to the result of a comparison operation between 1 `gwei` and 10^9 .

Tip: Look at how this is written for `gwei` and `ether` in the contract.

Check Answer **Show answer**

Next

Well done! No errors.

etherAndWei.sol

```
3
4 contract EtherUnits {
5     uint public oneWei = 1 wei;
6     // 1 wei is equal to 1
7     bool public isOneWei = 1 wei == 1;
8
9     uint public oneEther = 1 ether;
10    // 1 ether is equal to 10^18 wei
11    bool public isOneEther = 1 ether == 1e18;
12
13    uint public oneGwei = 1 gwei;
14    // 1 ether is equal to 10^9 wei
15    bool public isOneGwei = 1 gwei == 1e9;
16 }
```

Explain contract

0 ☐ Listen on all transactions

[vm] from: 0x5B3...eddC4 to: EtherUnits.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0x46c...3b76f

Tutorial 18: Transactions - Gas and Gas Price

LEARNETH 2.5.7 | Learneth Solidity Beginner Course

10.2 Transactions - Gas and Gas Price 18 / 19

Learn more about gas on ethereum.org.

Watch a video tutorial on Gas and Gas Price.

Assignment

Create a new `public` state variable in the `Gas` contract called `cost` of the type `uint`. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin `Gas Profiler` to check for the gas cost of transactions.

Check Answer **Show answer**

Next

Well done! No errors.

gasAndGasPrice.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Gas {
5     uint public i = 0;
6     uint public cost = 170367;
7
8     // Using up all of the gas that you send causes your transaction to fail.
9     // State changes are undone.
10    // Gas spent are not refunded.
11    function forever() public {
12        // Here we run a loop until all of the gas are spent
13        // and the transaction fails
14        while (true) {
15            i += 1;
16        }
17    }
18 }
```

Explain contract

0 ☐ Listen on all transactions

[vm] from: 0x5B3...eddC4 to: Gas.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0xf42...

Tutorial 19: Transactions - Sending Ether

The screenshot displays the Remix IDE interface during the completion of Tutorial 19: Transactions - Sending Ether. The interface is divided into several sections:

- Left Sidebar:** Contains a 'Tutorials list' with the current tutorial '10.3 Transactions - Sending Ether' selected. Below the list, there are instructions for building a charity contract and a 'Check Answer' / 'Show answer' button. A green message at the bottom states 'Well done! No errors.'
- Main Editor:** Displays the Solidity code for the 'Charity' contract. The code includes a constructor that sets the owner to the sender and two functions: 'donate' and 'withdraw'.
- Right Panel:** Shows the 'Explain contract' section with a search bar and a list of transactions. A single transaction is visible, showing a successful call to the 'Charity' constructor.

```
50 }
51 }
52 }
53 contract Charity {
54     address public owner;
55     constructor() {
56         owner = msg.sender;
57     }
58     function donate() public payable {}
59
60     function withdraw() public {
61         uint amount = address(this).balance;
62
63         (bool sent, bytes memory data) = owner.call{value: amount}("");
64         require(sent, "Failed to send Ether");
65     }
66 }
```

Transaction Log:

```
[vm] from: 0x5B3...eddc4 to: Charity.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0xf6e...78aea
```

Conclusion:

Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.